From Episodes to Revisable Concepts

The Experience Learning Layer for Evidence-Grounded Learning in Language Agents

Danny Ruchtie
Living working draft v0.5 - 9 August 2026
OPEN RESEARCH SPECIFICATION / EXPERIMENTAL PROTOCOL

Version 0.5 presents ELL as a paper-first open research project, adds a visual HTML reading edition and shared architecture diagrams, and narrows the repository to the manuscript, reproducible publication tools, schemas, evaluation cases, and executable research examples. The archived v0.1 PDF remains available for comparison.

Contents

Abstract

Large language model agents can store and retrieve previous interactions, yet retrieval alone does not establish that an agent has learned from experience. Learning requires a system to identify patterns across episodes, distinguish observations from hypotheses, form reusable concepts, apply those concepts in new situations, and revise them when later evidence disagrees. Recent agent-memory research has introduced reflection, experience-derived insights, workflow induction, dynamic memory graphs, semantic and procedural memory, schema induction, and temporally grounded updates. The remaining challenge is therefore not simply to add another memory store, but to make the transition from experience to general knowledge explicit, inspectable, revisable, and directly measurable.

This paper proposes the Experience Learning Layer (ELL), an open, model-independent architecture positioned between an agent's experience history and its decision process. ELL preserves raw episodes in an append-only store, creates typed associations between them, represents reflections as provisional interpretations, and promotes compatible reflections into versioned concepts. Each concept records its scope, supporting evidence, counterevidence, confidence, temporal validity, lineage, and observed utility. Concepts can be corroborated, contested, revised, superseded, or retired without erasing their evidential history.

The product north star is to give any AI a natural, evolving intuition about the user or organisation: grounded in experience, economical to use, sensitive to change, explainable when inspected, and always under user control.

Here intuition is neither anthropomorphism nor a new memory category. It names an emergent system capability produced by evidence-grounded compression, prediction, association, relevance selection, temporal adaptation, and outcome feedback. At use time, ELL should supply a compact intuition packet rather than dump raw history into a model, while retaining links that permit inspection, correction, deletion, and uncertainty-aware expansion.

The paper contributes a typed architecture, a concept-lifecycle protocol, an evidence ledger, an experimental benchmark for controlled concept induction, and a reproducibility plan based on openly licensed code and open- weight models. The central hypothesis is that separating reflection from concept consolidation, while preserving support and counterevidence, will improve transfer to structurally similar situations and reduce unsupported generalisation compared with episode-only retrieval or direct summarisation. The proposal is deliberately falsifiable: concept correctness, evidence coverage, revision behaviour, transfer, downstream task success, calibration, and cost are all measured independently.

Keywords: agent memory; experiential learning; reflection; concept formation; semantic memory; continual

learning; provenance; language agents; open source

1. Introduction

A language agent can remember an earlier event without learning anything general from it. A retrieval system may surface five examples of the same failure, yet the agent may still repeat that failure because the episodes have never been converted into a reusable and appropriately scoped principle. This distinction matters as agents move from single-turn assistance toward persistent collaboration, long-running tasks, and multi-session interaction.

The intended user experience is stronger than factual continuity. An AI should understand conversational shorthand, surface relevant context just in time, generalise usefully across related situations, notice when a previous pattern has changed, correct itself rapidly, and express calibrated uncertainty. Explanations should be available when requested without forcing provenance detail into every interaction. These behaviours constitute the proposed operational meaning of an evolving intuition; they do not imply consciousness, personhood, or a simulation of human cognition.

The first generation of agent-memory systems established that external memory could improve continuity beyond a model's context window. Generative Agents stored observations, ranked them for retrieval, and periodically synthesised higher-level reflections that influenced later plans (Park et al. 2023). Reflexion used textual feedback about prior attempts as a non-parametric learning signal (Shinn et al. 2023). ExpeL extracted natural-language insights from collections of task experiences and reused both insights and successful

trajectories at inference time (Zhao et al. 2024). MemGPT approached the problem as virtual context management, allowing an agent to move information between limited working context and external storage (Packer et al. 2023). CoALA organised these developments within a broader cognitive architecture containing working, episodic, semantic, and procedural memory (Sumers et al. 2024).

Subsequent systems have increasingly abstracted experience rather than merely retrieving it. Voyager accumulated reusable executable skills (G. Wang et al. 2023). Agent Workflow Memory induced recurring action routines from prior trajectories (Z. Z. Wang et al. 2025). A-MEM dynamically linked and revised memory notes as new information arrived (Xu et al. 2025). Reflective Memory Management used prospective and retrospective reflection to improve memory granularity and retrieval (Z. Tan et al. 2025). Temporal Semantic Memory consolidated temporally related events into durative representations (Su et al. 2026). More recent architectures explicitly construct schemas, concepts, semantic knowledge, or procedural knowledge from episodes, including DCPM, GAAMA, and PlugMem (Fei et al. 2026; Paul, Sharma, and Sareen 2026; Yang et al. 2026).

This rapid progress changes the research question. It is no longer defensible to claim that transforming episodes into abstractions is itself novel. The more precise question is: How should an agent maintain a transparent and revisable chain from raw experience, through provisional interpretation, to general concepts whose evidential support and behavioural usefulness can be tested?

ELL addresses this question by separating four objects that are often collapsed into one textual memory: 1. An episode records what occurred in a specific context.

2. A reflection is a provisional interpretation of one or more episodes.

3. A concept is a reusable proposition or strategy supported by multiple reflections and episodes.

4. An application outcome records whether using a concept helped in a later situation.

The separation is important because a plausible explanation is not automatically reliable knowledge. Suppose a project proposal is rejected after stakeholders were consulted late. A reflection may state that late stakeholder involvement caused the rejection. That is a useful hypothesis, but a single episode does not justify a universal rule. A concept should only be promoted after corroborating cases, explicit checks for counterexamples, and a scope statement such as “cross-functional initiatives with external implementation dependencies.” New evidence may later narrow, challenge, or replace it.

ELL treats every concept as a versioned claim rather than a timeless fact. The system retains the episodes that support it, episodes that contradict it, the contexts in which it has been applied, and the outcomes of those applications. This makes concept formation inspectable and enables evaluation at the level where learning is claimed to occur.

The architecture is inspired by, but does not attempt to reproduce, human memory. Tulving's distinction between episodic and semantic memory motivates the separation between event-specific records and general knowledge (Tulving 1972). Complementary Learning Systems theory motivates a fast episodic path and a slower consolidation path that integrates common structure across experiences (McClelland, McNaughton, and O'Reilly 1995; Kumaran, Hassabis, and McClelland 2016). Bartlett's account of reconstructive remembering provides a warning as much as an inspiration: abstraction can impose existing schemas on ambiguous experience and thereby distort it (Bartlett 1932). For this reason, ELL preserves provenance, counterevidence, and reversible revisions.

1.1 Scope

ELL is an external learning layer. It does not update the parameters of the underlying language model. It does not prescribe how raw activity is captured from calendars, files, screens, sensors, or chat applications. It starts after an application has produced a canonical stream of consented experience records. It is designed to support conversation histories, task trajectories, product-work records, and other event streams through a shared schema.

The initial research scope is deliberately narrower than a complete personal-intelligence system. It focuses on:

- association between episodes;
- reflection over individual and grouped episodes;
- concept formation and consolidation;
- application of concepts to later decisions;
- revision after new evidence and outcomes;
- direct evaluation of concept quality and transfer. Perception, multimodal capture, large-scale identity resolution, and parametric fine-tuning are outside the first paper.

1.2 Contributions

This working paper specifies five intended contributions.

A typed experience-to-concept architecture. It defines distinct schemas and operations for episodes, associations, reflections, concepts, evidence links, applications, and outcomes.

An evidence-grounded concept lifecycle. Concepts move through explicit states—proposed, corroborated, contested, revised, superseded, and retired—while retaining lineage and provenance.

A separation between confidence and eloquence. Concept confidence is computed from observable evidence, contradiction, diversity, and outcome history rather than accepted directly from an LLM's self- reported certainty.

A direct evaluation protocol. In addition to downstream task performance, the evaluation measures concept correctness, evidence precision and recall, scope accuracy, contradiction handling, revision latency, calibration, transfer, and cost.

An open reference implementation. Versioned releases will include code, schemas, prompts, experiment configurations, synthetic data generators, evaluation scripts, and paper sources under permissive licences, with reproducibility runs using open-weight models.

2. Problem Definition

2.1 Experience learning

Let an agent encounter an ordered experience stream E={e1 ,e2 ,…,en}.

Each episode ei contains a time-bounded record of context, observation, action, outcome, source, and metadata. Given this stream, an experience-learning system should produce a set of concepts C that improve decisions on future situations while remaining grounded in the episodes from which they were induced.

The goal is not to compress the entire history into a shorter text. Compression can preserve frequent information while losing exceptions, causal uncertainty, temporal change, and source identity. The desired output is a set of claims or strategies with explicit conditions and evidence.

A concept is useful only if it satisfies several properties:

- Grounded: its support can be traced to specific episodes and reflections.
- General: it applies beyond a single remembered case.
- Scoped: it states where it is expected to apply and where it may not.
- Revisable: contradictory evidence can change its status or content.
- Actionable: it can affect a later response, plan, or decision.
- Measurable: its correctness and utility can be evaluated independently.

2.2 Failure modes addressed

ELL is designed around seven common failure modes.

Episode replay without abstraction. Relevant cases are retrieved, but no reusable principle is formed.

Premature generalisation. A single salient episode becomes a broad rule.

Self-confirming reflection. Once a reflection is stored, later retrieval favours evidence that agrees with it.

Context collapse. A concept loses the conditions under which it was valid.

Temporal staleness. New evidence changes what is true, but the old concept continues to guide behaviour.

Provenance loss. A generated statement can no longer be connected to the experience that justified it.

Behavioural ambiguity. A system claims to have learned because it stored a summary, without showing improved transfer or decision quality.

2.3 Research questions

The first empirical study will answer five questions.

RQ1 — Separation. Does representing reflections as provisional objects before concept consolidation reduce unsupported generalisation compared with direct episode-to-rule summarisation?

RQ2 — Revision. Do explicit counterevidence, temporal validity, and versioned lifecycle states improve adaptation when the underlying pattern changes?

RQ3 — Transfer. Do consolidated concepts improve performance on new situations that share latent structure but differ in wording and surface details?

RQ4 — Efficiency. Can compact concepts plus selected source episodes match or improve downstream performance while using less retrieval context than episode-only methods?

RQ5 — Intuition quality. Can a compact, uncertainty-aware intuition packet improve shorthand understanding, just-in-time relevance, change detection, and rapid correction compared with no-memory, raw-history, and ordinary retrieval-augmented generation baselines?

2.4 Hypotheses

H1. ELL will produce higher concept-scope accuracy and lower overgeneralisation than a direct insight- extraction baseline.

H2. ELL will revise or contest invalid concepts more quickly after contradictory evidence than append-only reflection or rolling-summary baselines.

H3. ELL will yield a positive transfer gain on held-out tasks whose latent rule is represented in prior experiences, even when lexical similarity to those experiences is low.

H4. Retrieving concepts together with a small number of supporting episodes will reduce median input tokens per decision without reducing task success.

H5. Compact intuition packets will improve shorthand resolution, just-in-time context precision, change detection, and calibrated correction over no-memory, raw-history, and ordinary RAG baselines at an equal retrieval budget.

H6. ELL will reduce end-to-end decision tokens, latency, and measured hardware-time or energy proxies while preserving or improving task utility; this hypothesis includes the cost of background association, reflection, and consolidation rather than treating those operations as free.

These hypotheses are falsified if the confidence intervals include no practically meaningful improvement, if concept-level gains fail to translate into behaviour, or if the added architecture costs more than the retrieval it replaces without delivering measurable benefits.

3. Related Work and Positioning

3.1 Production memory and context infrastructure

Production-oriented systems establish useful implementation patterns without settling ELL's epistemic model.

MemGPT, now developed as Letta, treats a finite context window as a managed hierarchy and gives the agent explicit operations for moving information between tiers (Packer et al. 2023; Letta 2026). Mem0 provides a developer-facing memory layer that extracts, consolidates, and retrieves salient conversational information, with an optional graph representation (Chhikara et al. 2025). Zep and its open-source Graphiti engine maintain temporally valid entities and relations, preserve episode provenance, and combine semantic, lexical, and graph retrieval (Rasmussen et al. 2025; Zep 2026). TencentDB Agent Memory implements a local layered pipeline from raw conversations through atomic memories and scenes to personas, with inspectable files and hybrid retrieval (TencentDB Agent Memory Team 2026).

ELL adopts four principles from this family: explicit context-budget management; incremental temporal updates; hybrid candidate retrieval; and a drill-down path from compact context to source episodes. The corresponding integration points are replaceable ContextManager, MemoryProjection, TemporalRelationIndex, and RetrievalCandidateProvider ports. It rejects the assumption that any provider's extracted fact, persona, graph edge, summary, or context block is canonical truth. Such outputs enter ELL as untrusted candidates and remain subject to evidence, workspace, permission, version, deletion, and commit policy.

These projects are comparatively mature as deployable software, but their public evaluations use different models, prompts, budgets, datasets, and product configurations. Vendor- or project-reported latency, token, and accuracy results therefore motivate reproduction; they are not evidence that one substrate should be adopted as ELL's governing architecture. The current ELL implementation has not integrated or validated any of these systems end to end.

3.2 Reflection, reasoning memory, and non-parametric learning

Generative Agents introduced periodic reflection over accumulated observations, producing higher-level statements that could themselves be retrieved (Park et al. 2023). Reflexion showed that verbal feedback stored in episodic memory can improve repeated task attempts without weight updates (Shinn et al. 2023). ExpeL extended this idea by extracting insights across training experiences and transferring them to test tasks (Zhao et al. 2024). ReasoningBank distils strategies from both successful and failed trajectories and combines them with memory-aware test-time scaling (Ouyang et al. 2026). Reflective Memory Management used forward-looking summarisation and backward-looking evidence-based retrieval refinement (Z. Tan et al. 2025).

These systems establish reflection as a useful learning operator. ELL adopts that operator but makes a stricter distinction: a reflection is a candidate interpretation, not yet a trusted concept. This distinction enables explicit evidence thresholds, counterevidence checks, and lifecycle transitions. Self-judged success or failure is useful as one outcome signal but cannot be the sole authority for canonical learning; independent validators, task rewards, user corrections, and uncertainty must be retained where available.

3.3 Semantic, procedural, and graph-based abstraction

A second line of work converts experience into structured knowledge or reusable procedures. Voyager stores executable skills; Agent Workflow Memory induces reusable task routines; and A-MEM dynamically organises memory notes through contextual attributes and links (G. Wang et al. 2023; Z. Z. Wang et al. 2025; Xu et al. 2025). Temporal Semantic Memory aggregates temporally continuous evidence into durative user representations (Su et al. 2026).

The closest recent systems move explicitly from episodes to abstractions. DCPM separates fast fact updates from slower induction of schemas, intentions, and cross-domain patterns, with evidence links to supporting facts (Fei et al. 2026). GAAMA constructs a typed graph with episode, fact, reflection, and concept nodes (Paul, Sharma, and Sareen 2026). PlugMem derives semantic and procedural knowledge graphs from standardised episodic traces and maintains provenance to source episodes (Yang et al. 2026).

ELL is complementary to these architectures rather than a claim to precede them. Its intended distinction is the epistemic lifecycle of a concept. The core research object is not only a graph node or summary, but a claim with:

- separate supporting and contradicting evidence;
- an explicit applicability scope;
- a confidence calculation based on observable signals;
- temporal validity and version lineage;
- recorded downstream applications and outcomes;
- direct concept-level evaluation.

The implementation can use a graph, relational database, vector index, or a combination. The contribution is the contract and lifecycle, not a requirement for one storage technology.

3.4 Learned memory-operation policies

AgeMem exposes store, retrieve, update, summarise, and discard as tool actions and learns a unified short- and long-term memory policy with progressive reinforcement learning (Y. Yu et al. 2026). AtomMem decomposes management into atomic create, read, update, and delete operations and learns their orchestration with supervised and reinforcement learning (Huo et al. 2026). MemSkill instead represents extraction, consolidation, and pruning as reusable skills selected by a controller and revised by a designer that examines hard cases (H. Zhang et al.

2026).

ELL adopts the separation between atomic operations and policy, adaptive operation selection, and explicit learning from policy failures. These map to versioned MemoryOperationPolicy, ExtractionPolicy, ConsolidationPolicy, RetrievalPolicy, and ForgettingPolicy ports. A fixed heuristic and a learned policy should be interchangeable under the same typed inputs, outputs, budgets, and audit traces.

The rejected assumption is that task reward alone grants authority to mutate durable memory. Reward design can favour short-term benchmark success, opaque policies can make a write difficult to explain, and update or delete actions can destroy evidence needed for correction and scientific audit. ELL therefore permits learned policies to propose operations, but deterministic services enforce append-only provenance, authorization, idempotency, workspace isolation, tombstones, and canonical commit. These methods are recent research systems evaluated on bounded benchmark suites; their transfer, reward robustness, and deletion safety remain experimental questions.

3.5 Experience graphs and evolving skills

Experience-oriented systems increasingly represent reusable behaviour rather than only conversational facts.

EXG organises successful and failed trajectories into an experience graph for online growth and offline reuse (Jin et al. 2026). ReasoningBank and ExpeL distil general strategies across trajectories (Ouyang et al. 2026; Zhao et al. 2024). MUSE-Autoskill manages skill creation, storage, selection, evaluation, refinement, and skill-level experience under one lifecycle (Lin et al. 2026). Memento-Skills uses a learned router and reflective read-write loop to evolve structured external skills (Zhou et al. 2026).

ELL adopts graph-organised candidate generation, contrast between successes and failures, explicit skill lifecycle, and cross-task transfer measurement. A proposed SkillVersion should contain immutable identity, preconditions, scope, implementation or instruction content, lineage, tests, approval requirements, and rollback metadata. Each use should emit an ApplicationReceipt linking the exact skill and concept versions, retrieved evidence, action, cost, validator, and independently observed outcome. Outcome history may propose a new SkillVersion; it must not silently rewrite the version that was executed.

The rejected assumptions are that a skill file is self-validating, that repeated reuse proves causality, or that an LLM's reflection is an independent outcome measure. The named systems offer promising evidence in web, software-engineering, reasoning, and skill benchmarks, but do not yet establish universal transfer or safe autonomous modification. ELL therefore places ExperienceGraph, SkillGenerator, SkillRouter, and SkillEvaluator behind experiment ports and compares them under fixed token, latency, and validation budgets.

3.6 Neural and parametric test-time memory

Titans introduces a neural long-term memory module that learns to compress historical context and combines it with attention (Behrouz, Zhong, and Mirrokni 2025). Nested Learning interprets models and optimizers as nested optimization problems with distinct context flows; its HOPE architecture combines a self-modifying sequence model with a continuum memory system (Behrouz et al. 2025). TMEM distils experience into online LoRA updates so that a fast parameter state changes subsequent behaviour within an episode (Ren et al. 2026).

These approaches suggest promising accelerators for long context, fast adaptation, and policy specialization.

ELL may evaluate them through NeuralMemoryAccelerator or ParametricAdaptation ports while separately recording the source data, training objective, base model, update sequence, parameter delta, evaluation result, and deletion obligations. They are not suitable as the sole canonical store: a parameter update does not natively provide statement-level provenance, deterministic correction, workspace isolation, selective export, or reliable evidence-aware deletion. Their current evidence is primarily model- and benchmark-level; reproducibility, catastrophic interference, privacy erasure, and auditability remain release gates.

3.7 Evaluation of memory and intuition

LongMemEval evaluates information extraction, cross-session reasoning, temporal reasoning, knowledge updates, and abstention in long-term interactive memory (Wu et al. 2025). MemoryAgentBench adds incremental multi-turn evaluation of accurate retrieval, test-time learning, long-range understanding, and selective forgetting (Hu, Wang, and McAuley 2025). MemBench broadens factual and reflective memory evaluation across interaction roles (H. Tan et al. 2025), while MemoryArena couples memory with later action in multi-session environments (He et al. 2026).

ELL will use these benchmarks for comparability, but none alone establishes the intended intuition capability.

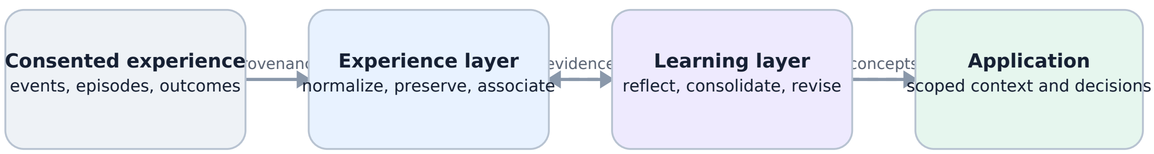
An ELL-specific sealed benchmark must additionally measure evidence-supported compression, shorthand resolution, proactive relevance, structurally distant transfer, calibrated uncertainty, change-point detection, correction latency, version and deletion lineage, and whether applying a retrieved concept or skill improved an independently measured outcome. Results must include background consolidation cost and compare fixed, learned, graph, external-memory, and neural accelerators under equal budgets.

3.8 Cognitive inspiration and its limits

The episodic-semantic distinction and complementary learning systems provide useful engineering metaphors (Tulving 1972; McClelland, McNaughton, and O’Reilly 1995; Kumaran, Hassabis, and McClelland 2016).

They suggest preserving specific experiences while separately integrating common structure. However, ELL is not a neuroscientific model. LLM-generated natural-language concepts, database records, and scheduled consolidation jobs are functional approximations. Biological terminology is used to clarify roles, not to claim mechanistic equivalence.

4. Experience Learning Layer Architecture



Where the Experience Learning Layer sits. ELL turns consented experience into evidence-backed, revisable concepts, then uses observed outcomes to improve or challenge them.

ELL is organised as a write–associate–reflect–consolidate–apply–evaluate loop. Each component exposes a narrow interface so that models, stores, and retrieval methods can be replaced independently.

The architecture separates three concerns. The input and capture layer normalises consented chats, recordings, documents, tool traces, and connectors into immutable sources, events, and episodes. Replaceable memory and retrieval infrastructure stores or indexes projections for association and candidate generation. The canonical learning layer governs hypotheses, concepts, applications, outcomes, versioning, permissions, and deletion.

External memory services and indexes can propose candidates, but they do not define canonical truth, identity, policy, or learning state.

4.1 Canonical episode store

The input is a canonical episode rather than an application-specific chat message. An episode is represented as $e_i=(id_i,t\_event_i,t\_observed_i,x_i,o_i,a_i,y_i,s_i,p_i)$, where id_i is the episode identifier, t_event_i and $t\_observed_i$ are event and observation time, x_i is context, o_i is an observation, a_i is an action, y_i is an outcome, s_i is source metadata, and p_i contains privacy and access-control metadata. The two timestamps allow delayed reports and later corrections to be represented.

The raw episode store is append-only. Corrections create new records linked by relations such as corrects, supersedes, or retracts. This preserves an audit trail and prevents an abstraction process from silently rewriting its own evidence.

4.2 Association layer

The association layer creates typed links between episodes. Initial relation types are:

- semantic similarity;
- shared entities or participants;
- temporal proximity or sequence;
- shared goal or task;
- shared action pattern;
- similar outcome;
- candidate causal relation;
- contradiction or correction. Only some edges are objective. Temporal sequence and shared identifiers can be deterministic; causal and contradiction edges may be model-generated hypotheses. Every edge therefore records its method, score, model or rule version, and source evidence. The layer supports both local neighbourhood retrieval and clustering. Reflection should not operate only on the nearest embeddings because lexically distant episodes may share a goal, failure pattern, or outcome. Hybrid candidate generation combines semantic, structural, temporal, and outcome-based signals.

4.3 Reflection Engine

A reflection is a provisional interpretation: $r_j=(h_j,t_j,s_j,p_j,n_j,u_j,z_j)$.

Here h_j is a claim or question, t_j is reflection type, s_j is scope, p_j and n_j are supporting and counterevidence episode sets, u_j is uncertainty metadata, and z_j is review status.

Reflection types include:

- observation or recurring pattern;
- causal hypothesis;

- strategy or procedural lesson;
- preference hypothesis;
- anomaly or exception;
- unresolved question;
- candidate correction to an existing concept.

The Reflection Engine is triggered by configurable events: a cluster reaching a minimum size, a repeated failure, a surprising outcome, new contradiction, elapsed time, or an explicit request. It produces structured output and is followed by an evidence-validation pass. Reflections may remain unverified, be marked supported, or be rejected before they reach the Concept Engine.

4.4 Concept Engine

A concept is a reusable, versioned proposition or strategy: c_k^(v)=(q_k,s_k,a_k,i_k,p_k,n_k,g_k,t_k,v,z_k).

Here qk is the proposition, sk is scope, ak is a set of applicability conditions, ik is the expected implication or recommended behaviour, pk and nk are supporting and counterevidence sets, gk is confidence, tk is temporal validity, v is version, and zk is lifecycle state.

Concept types include descriptive patterns, user preferences, causal hypotheses, strategies, constraints, and procedural rules. The first implementation focuses on textual concepts, but the schema permits executable procedures or references to code artefacts.

The Concept Engine performs five operations: 1. Propose: create a candidate concept from compatible reflections.

2. Merge: combine concepts that express the same claim and scope.

3. Split: separate an over-broad concept when evidence supports distinct conditions.

4. Revise: create a new version with changed claim, scope, validity, or confidence.

5. Retire or supersede: stop active use while preserving lineage.

Promotion requires evidence from more than one episode unless a domain-specific policy explicitly permits single-source facts. Evidence diversity is measured across time, source, participant, task, and surface form to reduce duplicate episodes being mistaken for independent support.

4.5 Concept registry and evidence ledger

The concept registry contains the current active version of each concept and its full version chain. The evidence ledger is append-only and records:

- which episodes supported or contradicted a reflection;
- which reflections contributed to a concept version;
- which model, prompt, rule, or human action produced a change;
- which concepts were retrieved for a decision;
- what action was taken and what outcome followed;
- deletions, access changes, and invalidations. This ledger supports explanation and reproducibility. A user or evaluator should be able to ask: “Why does the system believe this?”, “What evidence disagrees?”, “When did this change?”, and “Did using it help?”

4.6 Learning retrieval and application

At decision time, the retrieval service selects a mixture of concepts and source episodes. Concepts provide compact general guidance; episodes provide specificity, exception handling, and verification. The retrieval policy must be able to return either type independently.

A concept relevance score can initially be expressed as $R(c,q)=w1\ S1+w2\ S2+w3\ S3+w4\ S4+w5\ \mathrm{confidence}(c)-w6\ S6$,

where S1 through S4 represent semantic relevance, scope match, temporal validity, and observed utility; confidence(c) is concept confidence; and S6 is active contradiction. This formula is a design heuristic, not a validated scientific result. Weights will be tuned only on development data and frozen before test evaluation.

The retrieved package includes the concept statement, scope, confidence, current state, relevant support, relevant counterevidence, and source links. At the product boundary this compact, budgeted package is called an intuition packet. It may combine current concepts, selected episodes, procedures, open commitments, uncertainty, and known contradictions. It is a read object, not a new kind of memory. Applications can require a minimum confidence or allow contested concepts to be shown as uncertainty rather than instructions.

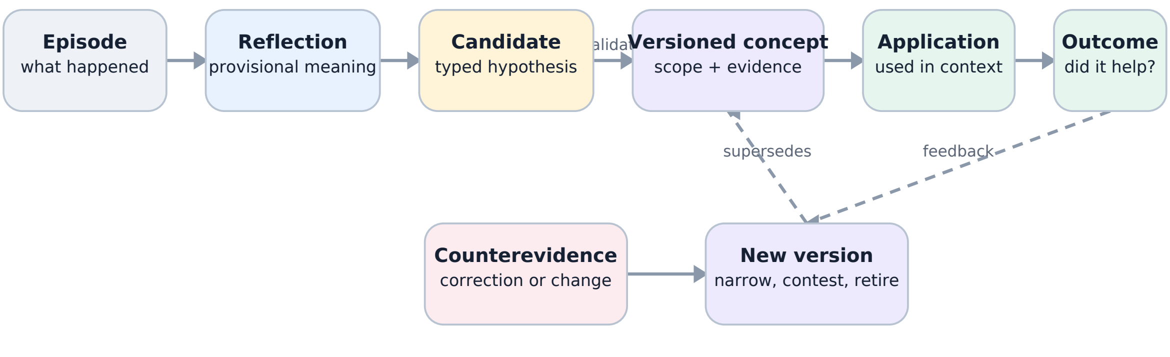
Formally, an intuition packet for query q, time t, policy context p, and budget b is $I(q,t,p,b)=(C^*,E^*,A^*,U^*,X^*,L^*)$, where C^* is the selected set of current concepts or hypotheses, E^* is the minimum restored source evidence, A^* contains applicable procedures or prospective commitments, U^* is calibrated uncertainty, X^* is material counterevidence or conflict, and L^* is provenance and selection lineage. The packet is valid only if every item satisfies p, its estimated context cost does not exceed b, and its lineage resolves to permitted canonical records.

4.7 Outcome loop

Every use of a concept creates an application record: $am=(xm,Cm,Em,dm,ym,um)$.

Here xm is the task context, Cm and Em are the concepts and episodes used, dm is the decision, ym is the observed outcome, and um is utility. Outcome feedback can come from task reward, user correction, validator output, or later observed consequences. The system must not automatically treat every positive result as proof of causality; outcomes are recorded as validation signals with their own reliability. Repeated success can strengthen a concept, while failure can add counterevidence, narrow scope, or trigger revision.

4.8 Lifecycle



From one episode to a revisable concept. Interpretation remains provisional until evidence, scope, policy, and counterevidence support promotion into a versioned concept.

The lifecycle is intentionally reversible. Corroborated does not mean permanently true. Contested concepts may still be retrieved when their uncertainty is relevant. Superseded concepts remain available for historical questions, while retired concepts are excluded from ordinary guidance.

5. Proposed Algorithms

5.1 Reflection scheduling

Running reflection after every episode would be expensive and may create redundant interpretations. ELL therefore uses event-based scheduling. An episode or cluster enters the reflection queue when one or more conditions are met:

- the cluster contains at least n distinct episodes;
- a failure or unusually high reward occurs;
- a new episode contradicts an active concept;
- association novelty exceeds a threshold;
- the concept has not been reviewed within a time window;
- a user or agent explicitly requests reflection. The scheduler records why a job was triggered. This enables later analysis of whether a scheduling policy created useful concepts or unnecessary model calls.

5.2 Reflection generation and critique

The Reflection Engine receives a bounded evidence packet: selected episodes, existing nearby reflections, relevant active concepts, and an instruction to produce typed hypotheses. The generator must output structured fields rather than a free-form essay.

A separate validation pass performs four checks: 1. Entailment: Does each cited episode actually support the claim attributed to it?

2. Contradiction: Are there nearby episodes that disagree?

3. Scope: Is the claim broader than its evidence?

4. Novelty: Is the reflection meaningfully different from existing reflections or concepts?

The validator can be an independent model, a deterministic rule, a human reviewer, or a combination. Using a separate model call does not guarantee independence, so experiments will include same-model and cross- model validation conditions.

5.3 Concept proposal

Reflections are grouped by semantic compatibility and scope overlap. A proposed concept requires:

- at least m compatible reflections;
- at least d distinct supporting episodes;
- minimum evidence diversity;
- no unresolved high-confidence contradiction;
- an explicit scope and exception list;
- a concise proposition and expected implication. The default policy will require support from at least three episodes across at least two distinct contexts. This is a configurable research parameter rather than a universal rule. A simplified proposal procedure is: for each reflection

cluster R: evidence = union(supporting episodes in R) counter = retrieve plausible counterevidence candidate = generate_concept(R, evidence, counter) checks = validate(candidate, evidence, counter) if checks pass promotion policy: register(candidate, state="proposed")

5.4 Evidence-weighted confidence

LLMs are not assumed to be calibrated judges of their own abstractions. ELL computes an operational confidence score from external features:

confidence(c)=1/(1+exp(-z_c)) , where zc is a weighted score that increases with supporting evidence, evidence diversity, and observed application utility, and decreases with counterevidence, age without revalidation, and validator disagreement.

The formula is deliberately transparent and will be calibrated on held-out development data. A simpler Bayesian or rule-based alternative will be included as a baseline. Confidence is never used to hide counterevidence; the raw counts and links remain visible.

5.5 Revision and temporal validity

When new evidence conflicts with a concept, the system chooses among four responses:

- add exception: the core claim remains valid but has a narrower boundary;
- narrow scope: the concept was over-generalised;
- change validity interval: the pattern was true during one period but is no longer current;
- replace claim: a new concept version better explains the evidence. Revision produces a new immutable version linked by revises or supersedes. The old version retains its former validity interval and applications. This design supports historical reasoning and avoids treating changed preferences or conditions as contradictions that must erase the past.

5.6 Merge and split

Duplicate concepts increase retrieval noise, while over-broad concepts hide meaningful exceptions. Merge candidates are identified using proposition similarity, overlapping evidence, compatible scope, and similar implications. Split candidates are triggered when counterevidence clusters around a coherent condition or when application outcomes differ significantly by context.

Both operations require a lineage record. A merge creates a new concept that references all parents. A split creates child concepts whose evidence partitions are explicit. Automated operations above a risk threshold can require human approval.

5.7 Retrieval with evidence restoration

Concept retrieval follows two stages. The first stage selects concepts by semantic, structural, temporal, and utility signals. The second restores a small, query-specific set of source episodes. This prevents a compact concept from becoming an ungrounded instruction.

The retrieval budget is fixed in tokens or characters so that systems are compared under equal context cost.

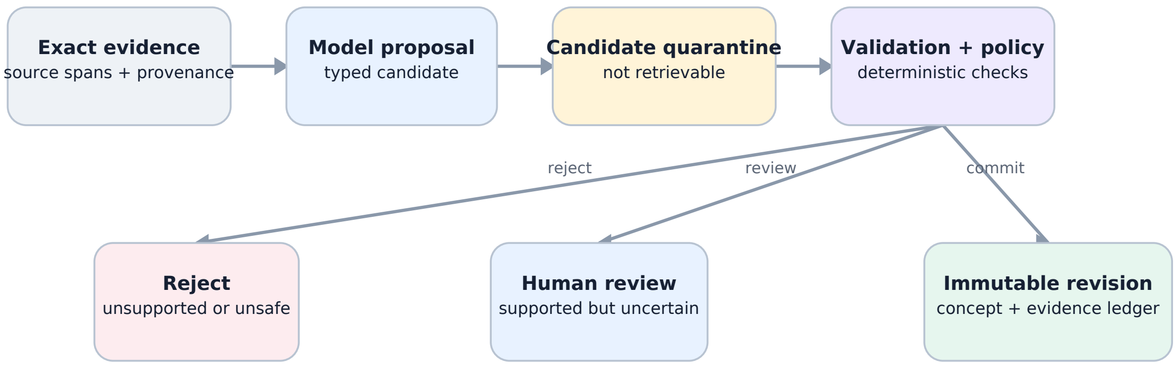
This avoids rewarding a method merely for retrieving far more text. Results report both task utility and memory information density.

5.8 Deletion and invalidation

A user must be able to delete source data. Deletion cannot stop at the episode store because derived concepts may still encode the removed information. ELL maintains derivation links so a deletion request can: 1. remove or tombstone the source episode according to policy; 2. remove it from support and counterevidence sets; 3. recompute affected confidence values; 4. contest, revise, or retire concepts that no longer meet support thresholds; 5. record the deletion cascade without retaining prohibited content.

This is both a privacy requirement and a scientific requirement: provenance is incomplete if derived claims cannot be invalidated when their evidence disappears.

6. Data Model and Public Interfaces



Models interpret; deterministic code governs. Generated interpretations enter quarantine. Schema validation and deterministic policy decide whether they are rejected, reviewed, or committed.

6.1 Core entities

Entity Purpose Required fields Episode Immutable record of a bounded experience ID, event time, observed time, context, observation, action, outcome, source, access metadata Association Typed relation between episodes or knowledge objects source ID, target ID, relation type, score, method, provenance Reflection Provisional interpretation or question statement, type, scope, support, counterevidence, uncertainty, status Concept Reusable versioned knowledge object proposition, scope, conditions, implication, support, counterevidence, confidence, validity, version, lifecycle state SkillVersion Proposed immutable reusable capability skill ID and version, preconditions, scope, implementation or instruction, tests, lineage, approvals, rollback metadata ApplicationReceipt Immutable record of concept or skill use task, exact concept and skill versions, retrieved episodes, decision, validator, cost, timestamp Outcome Evidence about application result reward or judgement, independent source, reliability, delay, linked application receipt AuditEvent Immutable record of system change actor, operation, object, prior version, new version, method, timestamp

6.2 Concept states

- proposed: generated and valid enough to test, but not yet strongly supported;
- corroborated: meets evidence and validation thresholds;
- contested: important unresolved counterevidence exists;

- revised: a new immutable version has been created in response to changed evidence and is awaiting or recording validation;
- superseded: replaced by a newer or more precise concept;
- retired: no longer eligible for ordinary retrieval;
- deleted: content removed under deletion policy, with only non-sensitive structural audit metadata retained where legally permitted. A revision never overwrites its parent. A revised version can later become corroborated, contested, superseded, retired, or deleted while its lineage remains traceable.

6.3 API contract

The reference implementation exposes the following model-independent operations: record_episode(episode) -> EpisodeID associate(episode_id, policy) -> list[Association] run_reflection(scope, trigger) -> list[Reflection] reconcile_concepts(reflection_ids) -> list[ConceptVersion] retrieve_learning(query, context, budget) -> LearningPacket record_application(packet, decision) -> ApplicationID record_outcome(application_id, outcome) -> list[ConceptUpdate] propose_skill(experience_ids, policy) -> list[SkillVersion] record_skill_application(skill_version_id, receipt) -> ApplicationID explain_concept(concept_id, version=None) -> EvidenceReport delete_subject_data(subject_id, policy) -> DeletionReport

LearningPacket is the central read object. It contains concepts, relevant evidence, conflicts, and budget accounting. It never contains only a naked concept statement.

6.4 Storage independence

The core schema is storage-neutral. The starter implementation uses an in-memory store; a SQLite adapter is planned as the first persistent baseline. Production adapters may add PostgreSQL, a vector index, or a graph database. Storage choices are evaluated by correctness, write cost, query latency, deletion support, and operational simplicity rather than by architectural fashion.

The same principle applies to model providers. The engine depends on a structured-generation interface, not a vendor SDK. Reproducibility experiments use open-weight models, while optional adapters may support hosted models for comparison.

Association, extraction, consolidation, retrieval, and forgetting policies may become configurable or learned, but their outputs remain proposals behind stable typed ports. Deterministic code continues to enforce provenance, workspace isolation, permissions, deletion, schema validity, and canonical commit rules. Competing lexical, vector, graph, temporal, probabilistic, and model-driven policies remain interchangeable experimental conditions until evaluation establishes which combination is useful. A Zettelkasten-style linked-note representation may be included as a simple baseline, but static note linking is too limited to govern temporal change, uncertainty, counterevidence, outcome feedback, and user control.

7. Experimental Design

7.1 Study status

The evaluation is preregistered in this paper before results are collected. All thresholds, prompts, model versions, random seeds, exclusions, and statistical tests will be committed before the sealed test sets are run.

Any later changes will be reported as deviations rather than silently folded into the method.

7.2 Evaluation stages

Stage A: controlled latent-pattern streams

The project will release a synthetic benchmark in which each experience stream is generated from known latent concepts. A stream contains:

- recurring contextual rules;
- paraphrased surface forms;
- irrelevant but semantically similar episodes;
- exceptions with explicit conditions;
- contradictory observations;
- temporal change points;
- delayed outcomes;
- duplicated or correlated evidence. Example latent concept: When a task depends on another team and the owner is not involved before commitment, delivery risk increases; the pattern does not apply to independently executable tasks. The generator creates episodes that support, contradict, or fall outside the concept's scope. The benchmark therefore has gold labels for the concept, its conditions, its evidence, counterevidence, and validity interval. Three stream lengths are planned: 50, 200, and 1,000 episodes. Difficulty increases through noise, lexical distance, exceptions, and concept drift. A sealed test generator seed prevents manual prompt tuning against the exact cases.

Stage B: long-term conversational memory

LongMemEval and LoCoMo will test whether ELL remains competitive on factual extraction, cross-session reasoning, temporal reasoning, updates, abstention, and long-range dialogue understanding (Wu et al. 2025;

Maharana et al. 2024). MemBench will add reflective-memory and efficiency measures where licensing and harness compatibility permit (H. Tan et al. 2025). MemoryAgentBench will test retrieval, test-time learning, long-range understanding, and selective forgetting in incremental multi-turn interaction (Hu, Wang, and McAuley 2025).

ELL is not expected to dominate span-recall tasks solely because it forms concepts. These benchmarks test whether the abstraction layer preserves ordinary memory performance and whether concept retrieval helps multi-session inference.

Stage C: memory-dependent action

MemoryArena will test whether information learned in earlier sessions improves later task execution (He et al.

2026). A smaller open environment may be added for rapid iteration, but MemoryArena is the target action benchmark because it explicitly couples memory, agent decisions, and environment outcomes.

Stage D: ELL intuition and outcome benchmark

The ELL-specific benchmark combines controlled organisational and personal-work streams with later tasks that require conversational shorthand, proactive but non-intrusive context, structurally distant transfer, adaptation at known change points, and explicit correction. Each decision records an ApplicationReceipt and an independently scored outcome. Gold data identifies required context, irrelevant private context, support, counterevidence, validity intervals, and deletion cascades. The benchmark therefore measures not only whether information was recalled, but whether a compact intuition packet selected the right evidence, improved behaviour, remained calibrated, and changed or forgot appropriately.

7.3 Memory Dynamics and Intuition Simulation Lab

The evaluation will include a reproducible simulation laboratory for studying memory dynamics before any policy is considered for deployment. The laboratory replays deterministic, timestamped scenario streams through the same typed ports used by the reference architecture. It supports parameter sweeps and side-by-side policy comparisons for extraction, association, consolidation, retrieval, revision, and forgetting without granting an experimental policy authority to mutate canonical evidence. Each run fixes the scenario version, configuration, model and prompt identifiers, random seed, permissions, and resource budget. It emits immutable decision and ApplicationReceipts that resolve every selected context item, candidate mutation, committed version, action, outcome, evaluator judgement, and cost to its permitted source evidence.

Scenarios use synthetic or appropriately de-identified longitudinal personas representing both individuals and organisations. They include stable preferences, weak signals, sensitive attributes that must not be inferred or revealed, explicit consent changes, contradictions, temporal drift, delayed outcomes, workspace boundaries, and deletion requests. Chronological train, development, and sealed test intervals prevent policies from learning from future events; change points and deletion events remain hidden from the system until they occur in replay. The laboratory compares no memory, raw-history or maximum-context prompting, ordinary RAG, and the candidate systems and ELL ablations in Section 7.4 under matched sources, model conditions, context budgets, and outcome opportunities.

Preregistered primary measures cover retrieval precision and recall; faithfulness and calibration of claims; adaptation latency after change or correction; contradiction and stale-memory rates; transfer and downstream task utility; overpersonalisation and unjustified inference; privacy, consent, and cross-workspace leakage; deletion- cascade completeness; and latency, token, storage, and energy or hardware-time cost, including consolidation.

Scoring is deterministic wherever gold state and event traces permit. Outcome and safety judgements are produced by evaluators independent of the policy under test; subjective conversational naturalness is assessed separately by blinded human reviewers. Stochastic conditions use multiple preregistered seeds and report confidence intervals and effect sizes. This laboratory is an evaluation proposal, not evidence that the present implementation already provides adaptive intuition or safe learned memory operations.

7.4 Baselines

At minimum, all experiments include: 1. no persistent memory; 2. full or maximum available context; 3. ordinary RAG or vector retrieval over raw episodes; 4. rolling summary memory; 5. linked-note or Zettelkasten-style retrieval; 6. episode retrieval plus direct insight extraction; 7. ELL without concept consolidation; 8. ELL without outcome feedback or temporal adaptation; 9. full ELL with compact intuition packets.

Where reproducible implementations and compatible licences are available, A-MEM, Reflective Memory Management, GAAMA, and PlugMem will be evaluated using their released code or carefully documented reimplementations (Xu et al. 2025; Z. Tan et al. 2025; Paul, Sharma, and

Sareen 2026; Yang et al. 2026). A method will not be included under another system's name if key behaviour cannot be reproduced.

Additional experiment tracks compare Graphiti/Zep, Mem0, Letta/MemGPT, and TencentDB Agent Memory as replaceable retrieval or context substrates; AgeMem, AtomMem, and MemSkill as learned operation policies; EXG, ReasoningBank, MUSE-Autoskill, and Memento-Skills as experience or skill learners; and Titans, HOPE, and TMEM as neural or parametric accelerators. A named comparison is reported only when its public interface or paper can be reproduced under the same source stream, model condition, context budget, and outcome metric.

7.5 Model conditions

The main reproducibility track uses at least two openly licensed instruction-tuned models from different model families and two capacity bands. Model names are recorded only when the experiment is frozen, because available open models change rapidly. All generation settings, quantisation, serving software, prompts, and hardware are logged.

A hosted-model comparison may be reported separately, but the paper's core claims must remain reproducible without proprietary APIs.

7.6 Metrics

Downstream metrics

- task success or benchmark accuracy;
- answer faithfulness to retrieved evidence;
- transfer gain over the episode-only baseline;
- abstention quality;
- action efficiency, including steps to completion. Concept metrics Concept correctness. Does the induced proposition match the gold latent pattern or human judgement? Evidence precision. What proportion of cited support actually supports the concept?

Evidence recall. What proportion of relevant support was linked?

Counterevidence recall. Did the system find important exceptions and contradictions?

Scope accuracy. Does the concept apply to the right contexts without over-generalising?

Revision latency. How many contradictory episodes or how much time elapses before a stale concept is contested or revised?

Lineage correctness. Do revisions, merges, and splits preserve accurate parent and evidence links?

Calibration. Does stated confidence correspond to observed concept correctness and application success?

Brier score and expected calibration error will be reported.

Efficiency metrics

- input and output tokens per episode and per decision;
- model calls per accepted concept;
- storage growth;
- median and tail latency;
- energy or hardware-time proxy where measurable;
- task utility per 1,000 retrieved tokens.

Product success criteria

- shorthand resolution accuracy without requiring the user to restate stable context;
- precision and recall of proactive just-in-time context, including a penalty for intrusive or irrelevant retrieval;
- useful generalisation to structurally related but lexically dissimilar situations;
- change-detection delay and correction latency after explicit or behavioural counterevidence;
- calibration and selective abstention when evidence is weak, stale, or contradictory;
- explanation faithfulness, citation validity, and user ability to correct, scope, or delete the underlying memory;
- end-to-end task utility per token, latency, storage, model call, and energy or hardware-time proxy, including background consolidation cost;
- zero cross-workspace leakage and complete tested invalidation of deleted evidence from derived projections.

Thresholds for these criteria will be frozen before choosing a production memory substrate or learned policy.

An implementation is not successful merely because it stores more, produces persuasive summaries, or retrieves plausible context.

7.7 Human evaluation

Concept quality cannot be reduced entirely to lexical matching. A stratified sample will be rated by at least three annotators who are blind to the system condition. The rubric covers correctness, support, scope, usefulness, and whether counterevidence was handled appropriately. Inter-rater agreement is reported using Krippendorff’s alpha. Disagreements are retained rather than resolved solely by an LLM judge.

LLM-based judges may scale evaluation, but they will be calibrated against the human sample. Prompts and raw judgements will be released. The same model used to generate a concept will not be the only judge of that concept.

7.8 Ablations

The following components are removed one at a time:

- reflection–concept separation;
- counterevidence retrieval;
- evidence diversity threshold;
- temporal validity;
- lifecycle versioning;
- application-outcome feedback;
- source-episode restoration at retrieval;
- graph associations beyond vector similarity. Additional sweeps vary reflection frequency, promotion thresholds, retrieval budget, and confidence formulation. Architecture ablations compare lexical, vector, graph, temporal, and hybrid association; fixed versus learned extraction and consolidation policy; concept-only versus episode-restored packets; and eager versus event-triggered background processing. This keeps dynamic graph organisation, probabilistic hypotheses, and other research directions behind evidence gates instead of presuming a settled winner. Safety ablations separately disable deterministic validation around learned write, update, and discard proposals; independent outcome validation around skill evolution; and canonical

evidence restoration around neural or parametric adaptation. These conditions are diagnostic and are not deployment configurations.

7.9 Statistical analysis

Binary outcomes use paired tests such as McNemar’s test where appropriate. Continuous or ordinal metrics use paired bootstrap confidence intervals and report effect sizes. Multiple random seeds are used for synthetic generation and stochastic inference. The primary comparisons and practical significance thresholds are frozen in the experiment configuration before the sealed tests.

Results are reported by task category and stream length, not only as a single average. Failure analysis separates write, association, reflection, consolidation, retrieval, reasoning, and application errors.

8. Open-Source Reference Implementation

8.1 Licensing

The repository is released under the permissive MIT License. It covers the paper source, diagrams, documentation, build tools, schemas, synthetic benchmark content, and reference code unless a file states otherwise. Third-party datasets retain their original licences and are downloaded separately. A future archival research release should additionally include machine-readable citation metadata and generated dependency attribution.

8.2 Repository structure

experience-learning-layer/ □□□ README.md □□□ LICENSE □□□ Makefile □□□ pyproject.toml □□□ paper/ □ □□□ ELL_Paper.md □ □□□ build_paper.py □ □□□ build_html.py □ □□□ diagrams.py □□□ docs/ □ □□□ index.html □ □□□ paper/ □ □□□ assets/diagrams/ □□□ src/ell/domain/ □□□ schemas/ □□□ evals/golden/ □□□ tests/ □□□ output/pdf/ The reference repository contains the living manuscript, generated HTML and PDF editions, shared visual diagrams, typed domain models, an in-memory governed kernel, deterministic evaluation utilities, golden cases, and tests. The code is an executable research example rather than an application or completed learning system.

8.3 Reproducibility requirements

Every reported experiment must include:

- immutable configuration files;
- exact Git commit;
- model identifiers and hashes where available;
- prompts and structured-output schemas;
- random seeds;
- hardware and serving configuration;
- raw outputs and failure logs;
- evaluation code and judge prompts;
- token, latency, and storage accounting. A one-command small-scale reproduction should run on consumer hardware with a compact open model. Full benchmark results may require larger hardware, but the same code path and data format must be used.

8.4 Contribution model

The project begins with a lightweight maintainer model and public design records. Significant architecture decisions use short decision documents in docs/decisions/. Contributions require tests, provenance for benchmark changes, and a declaration when generated data or code was produced with model assistance.

Research claims are reviewed separately from software changes. A faster implementation is not accepted as scientifically equivalent unless its behaviour and evaluation remain comparable.

8.5 Private data boundary

Private conversation exports can be used locally to test ingestion and qualitative usefulness, but they are not part of the public benchmark and must not be committed. Public experiments use synthetic or properly licensed data. The starter repository includes data-governance guidance; redaction hooks, local namespaces, and tested deletion cascades are required before any longitudinal pilot.

9. Ethics, Privacy, and Security

A memory system can improve continuity while also increasing risk. Persistent concepts may encode sensitive information, amplify a mistaken interpretation, or influence actions long after the originating interaction.

Long-term memory is therefore a control channel, not merely a convenience layer. Research on trustworthy memory search similarly shows that semantically related memory can still be inappropriate or unsafe for the current task (Zhang et al. 2026).

ELL adopts the following principles.

Visibility. Users should be able to inspect active concepts, evidence, uncertainty, and change history.

Correction. A user correction is stored as high-priority evidence and can trigger immediate contestation, but it should not silently erase historical state when historical reasoning is required.

Deletion. Source deletion propagates to derived concepts. Derived content must not survive simply because it was transformed.

Least access. Retrieval enforces source-level permissions and subject namespaces before semantic relevance is considered.

No hidden certainty. Contested or weak concepts are labelled. Confidence and evidence counts are not replaced by persuasive prose.

Poisoning resistance. New memories do not automatically become trusted rules. Promotion requires independent evidence and validators, and sensitive actions can require human approval.

Purpose limitation. Concepts formed for one domain are not automatically reused in another. Cross-domain transfer is an explicit, auditable operation.

The system may still infer sensitive traits that were never stated directly. The research implementation therefore defaults to local processing, excludes sensitive-trait inference from public examples, and treats inferred personal concepts as user-governed data. Future work must test whether users can understand and meaningfully control these abstractions.

10. Limitations and Threats to Validity

First, textual concepts favour knowledge that can be expressed as propositions or instructions. Tacit motor skills, perceptual expertise, and distributed representations may not fit this format. ELL may therefore complement rather than replace parametric or executable skill learning.

Second, provenance does not guarantee truth. Episodes can be incorrect, duplicated, biased, or malicious. A perfectly traceable concept may still be grounded in poor evidence. Source reliability and adversarial robustness require separate study.

Third, LLM-generated reflections may introduce causal stories that are plausible but unsupported. The validation and counterevidence stages reduce this risk but cannot eliminate it. Human evaluation remains necessary for high-stakes domains.

Fourth, the proposed confidence formula is heuristic. Evidence counts are not independent observations, and application outcomes may be delayed or confounded. Calibration must be empirical and domain-specific.

Fifth, synthetic benchmarks can overstate progress if their latent rules resemble the system's representation.

The controlled benchmark is useful for diagnosis but must be paired with natural conversations and agentic tasks.

Sixth, comparisons across memory systems are difficult because ingestion granularity, retrieval budget, model backbone, prompts, and judge choice can dominate results. The protocol therefore fixes budgets and reports complete configurations, but residual implementation differences remain.

Seventh, the architecture adds cost and operational complexity. The system is unsuccessful if the concept layer does not deliver enough transfer, safety, or efficiency to justify reflection, validation, storage, and governance overhead.

Eighth, the term intuition can encourage anthropomorphism or hide uncertainty behind fluent behaviour. In this paper it is only an operational label for measured context selection and adaptation. A compact packet may also compress away rare but important exceptions, so source restoration, counterevidence retrieval, and abstention must be evaluated explicitly.

Ninth, reduced prompt tokens do not necessarily reduce total energy or latency. Background embedding, association, graph maintenance, reflection, and consolidation can move cost out of the visible request path.

Efficiency claims therefore require end-to-end accounting over both write-time and read-time work.

Tenth, learned memory operations introduce reward misspecification and policy-opacity risks. A policy may improve benchmark reward by discarding inconvenient counterevidence, overfitting retrieval to the judge, or retaining sensitive information. Immutable sources and deterministic commit constraints reduce but do not eliminate this risk.

Eleventh, neural and parametric memory may improve adaptation while weakening inspectability and selective deletion. An external provenance ledger can record how an update was produced, but it cannot by itself prove which parameter encodes a fact or that a deletion has removed all influence. Such methods cannot satisfy ELL's canonical governance requirements without new verification techniques.

Finally, cognitive analogies are limited. Human episodic and semantic memory motivate the separation of roles, but ELL should not be interpreted as an account of biological memory or consciousness.

11. Expected Outcomes and Falsifiability

This work is intended to produce a testable system rather than an unfalsifiable architecture diagram. The primary claim would be weakened or rejected under any of the following outcomes:

- concept-level metrics improve but downstream transfer does not;
- direct insight extraction matches ELL on scope and contradiction handling at lower cost;
- concepts systematically omit exceptions that episode retrieval preserves;
- confidence remains poorly calibrated across model families;
- revision creates instability or catastrophic concept churn;
- open-weight models cannot generate sufficiently reliable structured reflections;
- privacy-preserving deletion cannot remove derived information without rebuilding the store;
- performance gains disappear under equal retrieval budgets.

Negative results remain useful. They can show that explicit concept objects are unnecessary for some task classes, that reflection should be triggered only by failures, or that evidence-grounded retrieval is more valuable than consolidation. The repository and benchmark are structured so that these alternatives can be tested without preserving the original hypothesis.

12. Research and Implementation Roadmap

Phase 0 — Specification and scaffold

- publish the paper draft, schemas, repository structure, and governance files;
- implement typed models and in-memory stores;
- provide deterministic baseline engines and unit tests;
- open public issues for unresolved design choices. Exit criterion: a contributor can record episodes, create a reflection, promote a concept, retrieve it with evidence, and run the test suite locally. Phase 1 — Controlled benchmark
- implement the latent-pattern stream generator;
- implement the Memory Dynamics and Intuition Simulation Lab with deterministic replay, policy sweeps, immutable receipts, and chronological train, development, and sealed test partitions;
- define gold concepts, evidence, counterevidence, exceptions, and change points;
- freeze metrics, policy comparison budgets, privacy and deletion tests, and the annotation rubric;
- publish baseline results for raw retrieval, rolling summary, and direct insights. Exit criterion: a complete replay produces concept, outcome, safety, and cost metrics with fixed seeds and reproducible receipts. Phase 2 — Reflection and Concept Engines
- add structured model adapters for local open-weight inference;
- implement validation, promotion, merge, split, and revision;
- add the evidence ledger and lifecycle reports;
- run component ablations. Exit criterion: ELL can be compared with baselines on the sealed synthetic test set. Phase 3 — External benchmarks
- integrate LongMemEval and LoCoMo;
- integrate MemoryAgentBench, MemBench, and MemoryArena where licences and compute permit;

- reproduce selected open memory baselines under equal budgets;
- compare learned memory-operation, experience-graph, skill-evolution, and neural-memory tracks behind the same provider-neutral ports and canonical governance boundary;
- complete human evaluation and calibration. Exit criterion: results can support or reject H1–H6 across at least one conversational and one action benchmark. Phase 4 — Paper completion
- replace this preregistration status with methods and results generated from tagged releases;
- add statistical analysis, failure cases, and limitations discovered in practice;
- release raw experiment artefacts and a reproducibility package;
- submit to an appropriate agent, NLP, HCI, or continual-learning venue.

13. Experience Learning Layer Expansion

The research architecture above asks whether evidence-grounded concepts improve transfer and future behaviour. This expanded architecture generalises the v0.1 scaffold into L: an open, local-first experience-learning layer usable by people, applications, and AI agents. The expansion does not replace the original empirical question. It makes the operational boundaries needed to test the hypothesis explicit.

L is not primarily a chatbot, note-taking application, Zettelkasten, vector database, or graph visualisation. Its north star is to give any AI a natural, evolving intuition about the user or organisation—grounded in experience, economical to use, sensitive to change, explainable when inspected, and always under user control. Its product loop is to capture an experience, preserve its source, extract typed candidates, decide under deterministic policy what may become durable, consolidate related memories without losing contradictions, retrieve the smallest useful context, observe outcomes, and revise future behaviour. Models interpret meaning and propose. Deterministic services validate, authorize, version, and commit.

Intuition is an emergent capability of this loop, not another row type or a claim that the system resembles a person. Evidence-grounded compression, prediction, association, relevance selection, temporal adaptation, and outcome feedback should together support shorthand, proactive just-in-time context, useful generalisation, rapid correction, change detection, calibrated uncertainty, and optional explanation. The delivery object is a compact intuition packet with sufficient evidence and uncertainty to be useful, plus stable references for deeper inspection.

13.1 System layers and technology boundaries

The product architecture has three independently evolvable layers:

- input and capture adapters turn chats, Recall recordings, documents, application events, and future connectors into immutable source artifacts, normalized events, and bounded episodes;
- memory and retrieval infrastructure provides replaceable persistence, search, association, graph, vector, or hosted-memory projections;
- the canonical learning layer evaluates evidence, maintains temporal and probabilistic hypotheses, governs durable concepts, records applications and outcomes, and controls revision, contradiction, and forgetting.

The first layer and the provider-neutral kernel have verified foundations. The second is not yet operational end to end in the current reference implementation: SQLite, TencentDB Agent Memory, hosted memory services, vector indexes, graphs, and TurboVec remain candidate adapters or projections to integrate and compare. The third has deterministic Phase 0 lifecycle contracts, but not yet the complete closed learning loop. This distinction prevents an external memory product from silently becoming L’s truth or policy authority.

The proposed direction is immutable evidence plus dynamic, temporal, probabilistic concepts and hypotheses. Extraction, association, consolidation, retrieval, and forgetting policies should be pluggable and may become model-driven or learned. Their proposals remain subject to deterministic provenance, permissions, workspace isolation, deletion, schema, and canonical commit governance. No fixed graph, vector method, note-linking scheme, or model family is selected as the final architecture before the success criteria and ablations in Section 7 are run.

13.2 Plural memory semantics

The v0.1 paper distinguishes episodes, reflections, concepts, applications, and outcomes. The broader product architecture retains these objects while distinguishing additional memory semantics that have different authority, lifetime, and retrieval rules:

- source memory preserves immutable artifacts and exact addressable spans;
- working memory holds expiring state for the current task or conversation;
- episodic memory represents bounded experiences, decisions, actions, and outcomes;
- semantic memory represents temporal assertions rather than timeless key-value pairs;
- preference memory represents scoped choices, exceptions, strength, and explicitness;
- procedural memory represents versioned workflows, preconditions, approvals, evidence, and rollback;
- prospective memory represents commitments, reminders, deadlines, and unresolved threads;
- relational memory represents evidence-backed temporal edges;
- reflective memory records uncertainty, freshness, usefulness, contradictions, and knowledge gaps;
- policy memory defines retention, consent, provider, sharing, and deletion boundaries and is never ordinary model context.

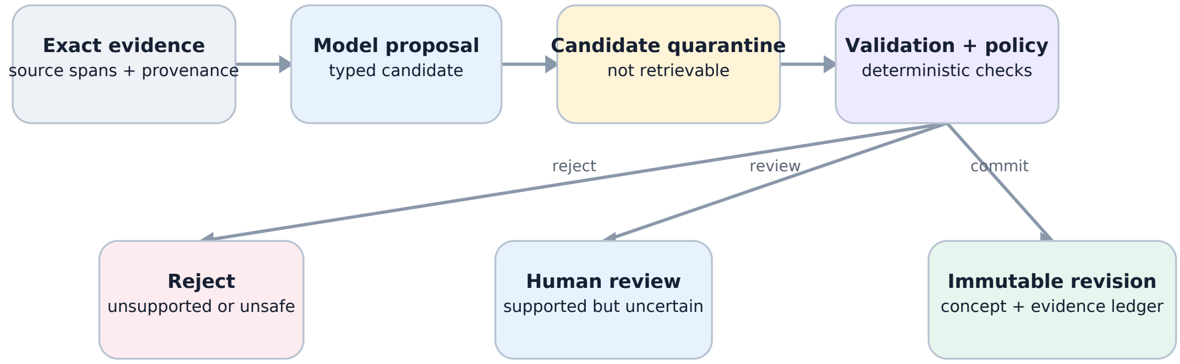
These forms may share infrastructure, but they must not be flattened into one untyped table with identical lifecycle semantics. The useful product is the explainable connection layer, not a decorative node cloud.

13.3 Source, event, and episode boundary

All provider connectors terminate at a common normalization boundary. An immutable SourceArtifact preserves source identity, checksums, timestamps, sensitivity, consent, and stable spans. Normalized ExperienceEvents represent user messages, assistant messages, tool calls and results, file changes, decisions, feedback, and external events. One or more events form a bounded Episode containing inputs, responses, actions, observations, and outcomes.

Stable deterministic identifiers make ingestion rerunnable. The same connector, external reference, source version, and source event produce the same domain IDs. Conversation exports, tool traces, calendar records, or future connectors can therefore be replaced without placing their schemas inside reflection, consolidation, policy, or retrieval services.

13.4 Governed candidate and commit path



Models interpret; deterministic code governs. Generated interpretations enter quarantine. Schema validation and deterministic policy decide whether they are rejected, reviewed, or committed.

A model response is untrusted structured input. It enters a CandidateMemory quarantine and cannot participate in normal retrieval. Deterministic validation checks the schema, cited source and span existence, workspace access, sensitivity inheritance, temporal fields, allowed predicates, and referenced memories. Policy then selects one of three initial outcomes:

- explicit user statements, corrections, and user-confirmed candidates may commit after validation;
- supported ordinary inferences and inferred procedures wait for review;
- unsupported non-explicit claims and sensitive model inferences are rejected.

The commit service is the sole canonical writer. It uses idempotency keys and optimistic concurrency, writes an immutable revision, appends a content-minimized audit event, and schedules disposable projections. A correction creates a new memory and closes the validity of the superseded revision. It never rewrites history. A lower-authority inference cannot supersede explicit user evidence.

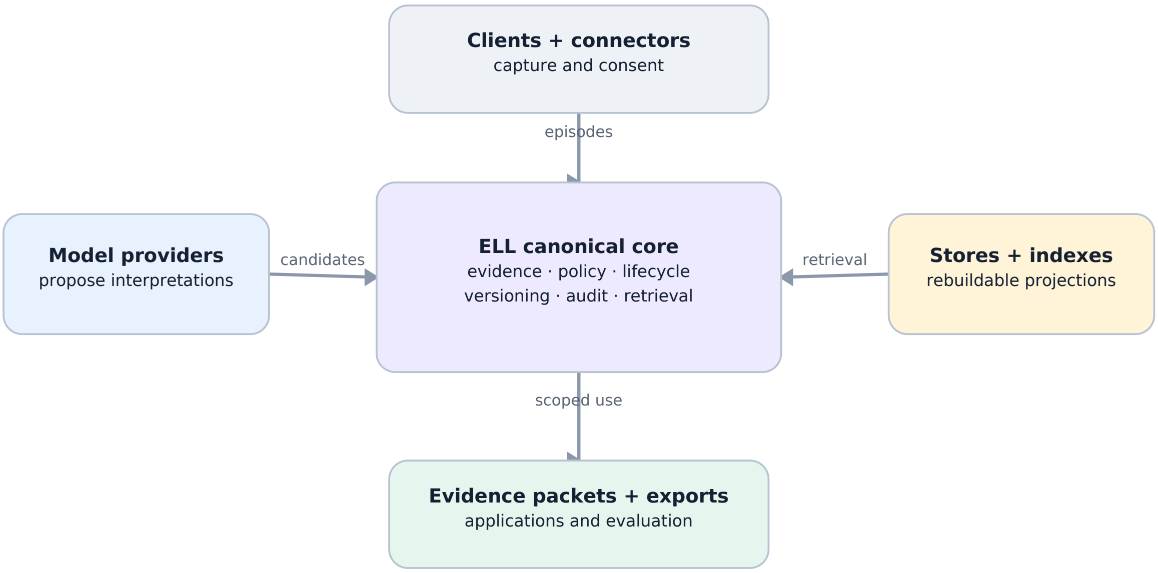
13.5 Policy-bound evidence retrieval

Retrieval is a typed service rather than direct database or vector-index access. A request includes actor, purpose, workspace, scope, allowed memory types, maximum sensitivity, evidence preference, and a fixed context budget. Authorization and lifecycle filtering happen before relevance ranking. Superseded, forgotten, expired, cross-workspace, and sensitivity-incompatible records do not enter ordinary results.

Candidate generation may combine lexical search, vectors, relation neighbourhoods, time, pinning, procedures, and later learned rerankers. Exact-vector, HNSW, graph, TurboVec, TencentDB Agent Memory, and hosted memory adapters remain replaceable projections to benchmark; no index or provider is canonical memory. The response is an EvidencePacket, presented at the product boundary as an intuition packet, containing selected current claims, scoped preferences, episodes, procedures, commitments, citations, selection explanations, uncertainty, and known material contradictions. A compact concept never becomes a naked instruction detached from its source.

Concrete comparison adapters may wrap Graphiti/Zep temporal graphs, Mem0 extraction and retrieval, Letta-style context management, or TencentDB Agent Memory's layered local store. Learned-policy adapters may implement AgeMem-style tool actions, AtomMem-style atomic operations, or MemSkill-style evolving routines. Experience and skill adapters may provide EXG-style graphs, ReasoningBank-style strategies, or lifecycle-managed skill candidates. Neural accelerators may provide Titans-, HOPE-, or TMEM-style state. In every case, the adapter returns candidates or acceleration state; current canonical SourceArtifact, CandidateMemory, and MemoryRecord contracts, together with proposed SkillVersion, ApplicationReceipt, and deletion records, remain governed by ELL.

13.6 Local-first and provider-neutral topology



Canonical learning stays provider-neutral. Clients, model providers, stores, and indexes are replaceable adapters. Canonical identity, evidence, policy, lifecycle, and audit remain inside ELL.

A complete usable state can live on one device. Network absence must not prevent capture, browsing, correction, or lexical retrieval. Remote processing visibly degrades to local processors or queued work. A practical reference adapter may use SQLite, content-addressed encrypted files, FTS5, and a replaceable vector index, but these technologies do not define the domain.

Four authentication concerns remain separate: L user identity, workspace authorization, model-provider credentials, and connector grants. A model adapter may call a provider API; a client or agent may call L through a narrow protocol; or L may export a scoped context bundle. Provider credentials never define L identity, canonical records, or access policy.

Forgetting immediately excludes a record, writes a tombstone, removes projections, and triggers evidence-aware invalidation. Sync must not resurrect tombstoned content. Sensitive trait inference is disabled for automatic durable learning. Imported documents are untrusted content, never system instructions. Secrets do not enter prompts, canonical memory, exports, analytics, or logs.

14. Reference Implementation Status

The repository now contains an executable Phase 0 proof aligned with the expanded architecture. This is implementation evidence, not an empirical result for H1-H6.

14.1 Versioned contracts and deterministic identity

Immutable Pydantic boundary models define SourceArtifact, SourceSpan, ExperienceEvent, Episode, CandidateMemory, MemoryRecord, AuditEvent, RetrievalRequest, and EvidencePacket. A registry exposes draft 2020-12 JSON Schemas with stable v1 identifiers and rejects unknown schema versions. Deterministic UUID derivation covers sources, events, and episodes so normalization can be rerun without changing identity.

14.2 Pure governed kernel

The LearningKernel runs without a database, network, or model. In-memory reference adapters implement artifact storage, candidate quarantine, immutable memory revisions, optimistic concurrency, idempotent mutation results, and append-only audit events. The deterministic policy and commit path enforce evidence, workspace, sensitivity, authority, correction, contradiction, forgetting, and retrieval lifecycle rules.

A fixture-backed DeterministicMockProvider advertises provider-neutral capabilities and validates every configured response against the requested Pydantic model. Missing or malformed fixtures fail loudly. It performs no network I/O and cannot invent a fallback response.

14.3 Golden evaluation corpus

The versioned synthetic corpus covers explicit scoped preferences, single-event inference, unsupported claims, sensitive inference, explicit correction, material contradiction, temporal change, Dutch evidence, and prompt injection embedded in imported content. Each JSONL case has a stable ID, exact evidence excerpt, typed candidate description, and expected policy or lifecycle outcome. Corpus validation rejects malformed or duplicate cases.

At this revision, the paper-first repository contains 31 passing kernel and contract tests plus publication tests for the generated HTML edition and shared diagrams. Strict static type checking passes for the Python domain source, and repository-wide Ruff lint passes. Product clients, databases, hosted services, provider SDKs, and vendor-specific memory scaffolding remain outside the repository and outside the canonical kernel.

14.4 Remaining gates

The in-memory proof does not yet demonstrate persistent-store rebuilds, deletion propagation through projections, encrypted sync, extension capability isolation, provider egress enforcement, job checkpoint recovery, or live multi-client operation. Those invariants are explicit deferred gates for future adapters. The next implementation step is the controlled benchmark: deterministic experience streams, sealed evaluation partitions, policy sweeps, application receipts, and reproducible comparisons with episode retrieval, rolling summaries, and direct insight extraction. The current implementation evidence is not a result for H1-H6.

15. Conclusion

The Experience Learning Layer begins from a simple distinction: access to old experience is not the same as learning from it. Current research already demonstrates reflection, dynamic memory organisation, schema induction, concept graphs, and semantic or procedural abstraction. The next useful contribution is therefore a stricter account of how an abstraction earns trust, remains connected to evidence, changes when challenged, and improves future behaviour.

ELL proposes an explicit path from episodes to provisional reflections, from reflections to versioned concepts, and from concepts to measured applications and outcomes. The architecture preserves both support and counterevidence, treats scope and temporal validity as first-class fields, and makes revision reversible and auditable. Its value will be determined empirically, not by the plausibility of the design.

The product expression of that hypothesis is an evolving intuition: economical, timely context that helps an AI understand shorthand and adapt to change without hiding where its guidance came from. Whether compact intuition packets deliver this experience more accurately and efficiently than no memory, raw history, ordinary RAG, rolling summaries, or dynamic-memory baselines remains an open empirical question.

The project is open by default. The paper, schemas, benchmark generator, implementation, prompts, and evaluation harness are intended to make the research easy to inspect, reproduce, criticise, and extend. The immediate next step is not to claim a finished learning system, but to build the controlled benchmark and reference implementation needed to test whether the proposed lifecycle actually works.

Acknowledgements

This working draft was developed as part of the open Experience Learning Layer project. Future versions will list contributors according to documented authorship and contribution criteria.

References

Bartlett, Frederic C. 1932. Remembering: A Study in Experimental and Social Psychology. Cambridge University Press.

Behrouz, Ali, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. 2025. ‘Nested Learning: The Illusion of Deep Learning Architectures’. https://arxiv.org/abs/2512.24695.

Behrouz, Ali, Peilin Zhong, and Vahab Mirrokni. 2025. ‘Titans: Learning to Memorize at Test Time’.

https://arxiv.org/abs/2501.00663.

Chhikara, Prateek, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. ‘Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory’. https://arxiv.org/abs/2504.19413.

Fei, Tianxiang, Mingyang Song, Mao Zheng, and Xiang Yu. 2026. ‘Memory Beyond Recall: A Dual-Process Cognitive Memory System for Self-Evolving LLM Agents’. https://arxiv.org/abs/2606.09483.

He, Zexue, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, et al. 2026. ‘MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks’. https://arxiv.org/abs/2602.16313.

Hu, Yuanzhe, Yu Wang, and Julian McAuley. 2025. ‘Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions’.

https://arxiv.org/abs/2507.05257.

Huo, Yupeng, Yaxi Lu, Zhong Zhang, Haotian Chen, and Yankai Lin. 2026. ‘AtomMem: Learnable Dynamic Agentic Memory with Atomic Memory Operation’. https://arxiv.org/abs/2601.08323.

Jin, Yuxin, Siyuan Zhang, Hanchen Wang, Lu Qin, Ying Zhang, and Wenjie Zhang. 2026. ‘EXG: Self-Evolving Agents with Experience Graphs’. https://arxiv.org/abs/2605.17721.

Kumaran, Dharshan, Demis Hassabis, and James L. McClelland. 2016. ‘What Learning Systems Do Intelligent Agents Need?

Complementary Learning Systems Theory Updated’. Trends in Cognitive Sciences 20 (7): 512–34.

https://doi.org/10.1016/j.tics.2016.05.004.

Letta. 2026. ‘Letta: Platform for Building Stateful Agents’. Software repository. https://github.com/letta-ai/letta.

Lin, Huawei, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. 2026. ‘MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation’. https://arxiv.org/abs/2605.27366.

Maharana, Adyasha, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. ‘Evaluating Very Long-Term Conversational Memory of LLM Agents’. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 13851–70. https://doi.org/10.18653/v1/2024.acl-long.747.

McClelland, James L., Bruce L. McNaughton, and Randall C. O’Reilly. 1995. ‘Why There Are Complementary Learning Systems in the Hippocampus and Neocortex: Insights from the Successes and Failures of Connectionist Models of Learning and Memory’.

Psychological Review 102 (3): 419–57. https://doi.org/10.1037/0033-295X.102.3.419.

Packer, Charles, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. ‘MemGPT: Towards LLMs as Operating Systems’. https://arxiv.org/abs/2310.08560.

Ouyang, Siru, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, et al. 2026. ‘ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory’. In International Conference on Learning Representations.

https://arxiv.org/abs/2509.25140.

Park, Joon Sung, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. ‘Generative Agents: Interactive Simulacra of Human Behavior’. In Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, 1–22. https://doi.org/10.1145/3586183.3606763.

Paul, Swarna Kamal, Shubhendu Sharma, and Nitin Sareen. 2026. ‘GAAMA: Graph Augmented Associative Memory for Agents’.

https://arxiv.org/abs/2603.27910.

Rasmussen, Preston, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. ‘Zep: A Temporal Knowledge Graph Architecture for Agent Memory’. https://arxiv.org/abs/2501.13956.

Ren, Tao, Weiyao Luo, Hui Yang, Rongzhi Zhu, Xiang Huang, Yuchuan Wu, Bingxue Chou, et al. 2026. ‘Scaling Self-Evolving Agents via Parametric Memory’. https://arxiv.org/abs/2606.04536.

Shinn, Noah, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. ‘Reflexion: Language Agents with Verbal Reinforcement Learning’. In Advances in Neural Information Processing Systems. Vol. 36.

https://arxiv.org/abs/2303.11366.

Su, Miao, Yucan Guo, Zhongni Hou, Long Bai, Zixuan Li, Yufei Zhang, Guojun Yin, et al. 2026. ‘Beyond Dialogue Time: Temporal Semantic Memory for Personalized LLM Agents’. https://arxiv.org/abs/2601.07468.

Sumers, Theodore R., Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2024. ‘Cognitive Architectures for Language Agents’.

Transactions on Machine Learning Research. https://arxiv.org/abs/2309.02427.

Tan, Haoran, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. ‘MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-Based Agents’. In Findings of the Association for Computational Linguistics: ACL 2025.

https://arxiv.org/abs/2506.21605.

Tan, Zhen, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long T. Le, Yiwen Song, et al. 2025. ‘In Prospect and Retrospect: Reflective Memory Management for Long-Term Personalized Dialogue Agents’. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, 8416–39. https://doi.org/10.18653/v1/2025.acl-long.413.

TencentDB Agent Memory Team. 2026. ‘TencentDB Agent Memory’. Software repository.

https://github.com/TencentCloud/TencentDB-Agent-Memory.

Tulving, Endel. 1972. ‘Episodic and Semantic Memory’. In Organization of Memory, edited by Endel Tulving and Wayne Donaldson, 381–403. Academic Press.

Wang, Guanzhi, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023.

‘Voyager: An Open-Ended Embodied Agent with Large Language Models’. In Transactions on Machine Learning Research.

https://arxiv.org/abs/2305.16291.

Wang, Zora Zhiruo, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. ‘Agent Workflow Memory’. In Proceedings of the 42nd International Conference on Machine Learning, 267:63897–911. Proceedings of Machine Learning Research.

Wu, Di, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. ‘LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory’. In International Conference on Learning Representations.

https://arxiv.org/abs/2410.10813.

Xu, Wujiang, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. ‘A-MEM: Agentic Memory for LLM Agents’.

In Advances in Neural Information Processing Systems. https://arxiv.org/abs/2502.12110.

Yang, Ke, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. 2026.

‘PlugMem: A Task-Agnostic Plugin Memory Module for LLM Agents’. https://arxiv.org/abs/2603.03296.

Yu, Yi, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. 2026. ‘Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents’. https://arxiv.org/abs/2601.01885.

Zhang, Jiawen, Kejia Chen, Jiachen Ma, Yangfan Hu, Lipeng He, Yechao Zhang, Jian Liu, Xiaohu Yang, Tianwei Zhang, and Ruoxi Jia.

2026. ‘Beyond Similarity: Trustworthy Memory Search for Personal AI Agents’. https://arxiv.org/abs/2606.06054.

Zhang, Haozhen, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. ‘MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents’. https://arxiv.org/abs/2602.02474.

Zhao, Andrew, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ‘ExpeL: LLM Agents Are Experiential Learners’. In Proceedings of the AAAI Conference on Artificial Intelligence, 38:19632–42. 17.

https://doi.org/10.1609/aaai.v38i17.29936.

Zhong, Wanjun, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. ‘MemoryBank: Enhancing Large Language Models with Long-Term Memory’. In Proceedings of the AAAI Conference on Artificial Intelligence, 38:19724–31. 17.

https://doi.org/10.1609/aaai.v38i17.29946.

Zhou, Huichi, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, et al. 2026. ‘Memento-Skills: Let Agents Design Agents’. https://arxiv.org/abs/2603.18743.

Zep. 2026. ‘Graphiti: Build Real-Time Knowledge Graphs for AI Agents’. Software repository.

https://github.com/getzep/graphiti.